Backend Requirements

What the web app needs from the API, storage and services to run end to end with real data.

Prepared for	Backend engineer
Prepared by	Samuel Olusanya (frontend), Gloxad Academy
Date	7 October 2026
Frontend	Next.js 14 app (repo: Gloxadapp). Browser calls /api/* on the web app, which proxies to the backend.
API reviewed	https://gloxard-backend.onrender.com (reference page at the backend root lists 76 routes)
Scope	A. Fix first (existing API and storage) | B. Institution partner module (new) | C. Trimester LMS (new) | D. Cross-cutting requirements | E. Delivery order and acceptance checks | Appendices

1. How to read this document

The web app now has three areas. The **main site** (courses, tutors, admin) already talks to your API. The **institution partner module** (landing page, onboarding, institution dashboard and the Gloxad reviewer portal) and the **trimester LMS** (student, instructor and admin views) are fully designed and built on the front end, but run on demo data because the endpoints they need do not exist yet. This document lists, screen by screen, what the API must provide, plus the problems in the existing setup that block real use.

Label	Meaning
BLOCKER	Existing problem that stops uploads, production use or testing. Fix first.
VERIFY	Route is documented on the API reference, but behaviour behind a login has not been tested from the front end.
NEW	Does not exist yet (returns 404 today). Needs design and build.
DECISION	Needs a product or engineering decision before it can be built.

What was checked on 7 October 2026

Read-only checks against the live API, without signing in:

- `GET /api/health` returns `{"status":"ok","uptime":...}`. This is process uptime only (see A7).
- Public routes work: `GET /api/categories` (3 categories, now with `imageUrl`) and `GET /api/courses` (paginated).
- Documented admin, tutor and auth routes answer **401** without a token, so they exist. Nothing behind a login has been exercised.
- The S3 bucket `gloxad-bucket` **does not exist** (S3 returns `NoSuchBucket`). All 7 image URLs the API returns (categories, course thumbnails, trailer) return 404.
- CORS allows only `http://localhost:3000`. `https://www.gloxadacademy.com` receives no CORS headers.
- No routes exist for institutions, cohorts, assignments, live sessions or the community forum (all return 404 `Route not found`).

Conventions the front end already relies on

- **Format:** JSON, camelCase fields, MongoDB `_id` strings, ISO-8601 UTC timestamps.

- **Auth:** `Authorization: Bearer <JWT>` (7-day tokens today). Roles in use: `student`, `instructor`, `admin`. New: `institution_admin` (see Part B).
- **Errors:** HTTP status plus `{ "error": "message" }`. Please add `{ "error": "...", "fields": { "email": "..." } }` for validation failures so forms can show field errors.
- **Lists:** `?page=1&limit=10` returning `{ <items>: [...], pagination: { total, page, limit, totalPages, hasNextPage, hasPrevPage } }`, matching `GET /api/courses` today.
- **Uploads:** the browser calls `POST /api/upload/presigned-url`, PUTs the file straight to S3, then saves the returned `fileUrl` on the resource.
- **Real time:** `socket.io` is already used for messaging (token passed as `auth: { token }`). The LMS live class reuses it.

Part A. Fix first: existing API and infrastructure

These items block uploads, production deployment or testing. None of the new modules can be completed end to end until A1, A2 and A5 are done.

ID	Status	Item	What is needed
A1	BLOCKER	S3 bucket missing	Create or restore the bucket and tell us its name and region. Re-seed the image files, or update the seeded URLs if the name changes. Apply the CORS and prefix rules below.
A2	BLOCKER	API CORS for production	Allow `https://www.gloxadacademy.com`, `https://gloxadacademy.com` and the staging origin. Today only `http://localhost:3000` is allowed.
A3	VERIFY	Category image on write	`GET /categories` returns `imageUrl`. Confirm `POST` and `PUT /admin/categories` accept and store `imageUrl` (the admin form sends `icon` as a name and `imageUrl` as the uploaded file).
A4	VERIFY	Token refresh contract	`POST /auth/refresh` is documented to accept the Bearer token or `{ "token" }`. Confirm it accepts a token that expired recently (needs a grace window) and returns `{ token, user }`.
A5	BLOCKER	Testing environment	Provide a staging API with seed data, test credentials for admin, instructor and student (shared securely), and an OpenAPI or Postman file. No signed-in call has been tested yet.
A6	DECISION	Field-name pairs	Responses now carry both spellings: `isVerified`/`isEmailVerified`, `bio`/`biography`, `socials`/`socialLinks`, `avatar`/`avatarUrl`. Pick one canonical name per pair and announce a removal date for the other. Also make `category` consistent (an id in `/admin/courses/pending`, a name in `/courses`).
A7	NEW	Real platform-health metric	`/api/health` reports process uptime. The admin analytics screen shows "Platform Health 99.9% uptime this month". See D8.

A1. Bucket layout and CORS

Current upload folders accepted by `POST /api/upload/presigned-url`: `avatars`, `courses`, `certifications`, `resources`. The new modules need these as well. Public prefixes can be read by URL; private prefixes must only be served through short-lived signed GET URLs.

Prefix	Used for	Access	Limits (proposed)
`avatars/`	Profile photos	Public read	5 MB, image/*
`categories/`, `courses/`	Category images, thumbnails	Public read	10 MB, image/*
`courses/` (videos)	Lesson and trailer video	Signed or CDN with token	Large; use multipart upload
`certifications/`	Tutor credential files	**Private**	10 MB, pdf/image
`logos/`	Institution logos NEW	Public read	2 MB, svg/png/jpg/gif, max 800x800 px
`signatures/`	Certificate signature NEW	Public read	1 MB, png
`documents/`	Partner application documents NEW	**Private**	10 MB, pdf
`submissions/`	Assignment files NEW	**Private**	50 MB, zip/pdf/source
`certificates/`	Issued certificate PDFs NEW	Private, signed GET	Generated by the server

Bucket CORS (browser uploads go straight to S3, so the bucket itself needs this, separately from API CORS):

```
[
  {
    "AllowedOrigins": ["http://localhost:3000", "https://www.gloxadacademy.com", "https://gloxadacademy.com"],
    "AllowedMethods": ["PUT", "GET", "HEAD"],
    "AllowedHeaders": ["Content-Type", "x-amz-*"],
    "ExposeHeaders": ["ETag"],
    "MaxAgeSeconds": 3000
  }
]
```

Presigned PUT URLs should also enforce the content type and a `content-length-range` per prefix, so the limits above hold even if the client is bypassed.

Part B. Institution partner module NEW

Front-end screens: partner landing page, activation, six-step onboarding wizard (logo, departments, tutors, students, subscription, branding), "Application Submitted", institution dashboard, and the Gloxad reviewer portal (dashboard, applications table, review drawer, approval modal).

B0. Flow DECISION

> **Decision needed before building.** The designs disagree about order. The onboarding wizard as designed comes first and ends in "Application Submitted, our team will review within 24-48 hours". The reviewer portal, however, says that approving an application "automatically sends an onboarding invitation email", which implies review comes first.

Recommended flow (needs your agreement; the front end will follow whatever is chosen):

1. Representative submits a **partnership request** (institution, type, website, address, expected students, requested features, message, documents). Creates an application with status `pending` and a reference ID such as `GLX-2026-9012`. Confirmation email sent. (This request form is not designed yet.)
2. Gloxad staff review in the reviewer portal: approve, reject or request more information.
3. **Approval** creates the institution and emails the representative a single-use activation link.
4. Representative **activates** (sets a password, minimum 12 characters), then completes the onboarding wizard. Each step saves to the server so it can be resumed on another device.
5. Finishing the subscription step activates the workspace and opens the institution dashboard.

B1. Data model

Entity	Key fields
InstitutionApplication	`referenceId`, `institutionName`, `type` (Private University, Vocational School, Charter School, Medical Institute, Public College, International School, Technical College, Business School, Law School, Online University, and so on), `website`, `address`, `representative{ name, role, email, phone }`, `expectedStudents` (range text and a numeric estimate), `requestedFeatures`, `message`, `documents[{ name, key, size, verifiedAt }]`, `status` (`pending`, `under_review`, `approved`, `rejected`), `plan` (`Premium`, `Enterprise`), `internalNotes`, `submittedAt`, `reviewedBy`, `updatedAt`
ReviewEvent (audit)	`applicationId`, `actorId`, `action` (status_change, plan_change, note, approve, reject, request_info, invite_sent), `from`, `to`, `createdAt`. The drawer shows "Last updated by Admin_Reviewer_2 on Oct 16, 11:45 AM".
Institution	`name`, `slug`, `logoUrl`, `branding{ displayName, color, tagline }`, `plan`, `billing` (`monthly`/`annual`), `status` (`pending_activation`, `onboarding`, `active`, `suspended`), `adminUserId`, `applicationId`, `onboarding{ currentStep, completedSteps[] }`, `seatLimit`, `createdAt`
Department	`institutionId`, `name` (required), `faculty`, `code`, `headName` or `headUserId`, `description`
InstitutionMember	`institutionId`, `role` (`tutor`/`student`), `name`, `email` (unique per institution), `departmentIds[]`, `designation` (tutors), `autoInvite`, `inviteStatus` (`not_invited`, `pending`, `accepted`, `expired`), `userId` once registered
Subscription / Payment	`institutionId`, `plan`, `billing`, `currency`, `amountGross`, `discount`, `amountTotal`, `taxAmount`, `status`, `provider`, `providerReference`, `periodStart`, `periodEnd`, `invoices[]`
PartnerStaff	Reviewer accounts with scoped permissions (nav item "Staff Management"; "Manage Staff Access: configure permissions for regional academy managers"). No screen is designed yet.

B2. Endpoints

Public and activation (base path `/api/institutions`)

Endpoint	Request	Response and rules
POST `/applications` Public. Rate-limit, CAPTCHA advised.	`institutionName*`, `type*`, `website`, `address{ line1, city, country }`, `representative{ name*, role*, email*, phone }`, `expectedStudents`, `requestedFeatures`, `message`, `documentKeys[]` (from presigned upload)	201 `{ application: { _id, referenceId, status:"pending", submittedAt } }`. Sends confirmation email. Reject duplicate pending applications for the same representative email.
GET `/applications/:referenceId` Public with emailed access code.	Query: `code`	`{ referenceId, status, submittedAt, estimatedReviewHours:48 }` for the "Track Request" button. No personal data.
GET `/plans` Public.	-	`{ plans: [{ key, name, tagline, monthlyPrice, currency, limits{ students, departments }, features[] }], annualDiscountPercent:20 }`. Single source of truth for prices (see B3).
POST `/activate` Public, single-use token.	`{ token, password }`	200 `{ token, user, institution }` (signed in). 400 weak password (min 12), 410 token used or expired (7 days).

Institution admin (base path `/api/institutions/me`; role `institution_admin`, scoped to their own institution)

Endpoint	Request	Response and rules
GET `/`	-	Institution plus `onboarding{ currentStep, completedSteps[] }` so the wizard resumes.
PUT `/branding`	`{ displayName, color (#hex), tagline, logoUrl }`	Updates branding. `logoUrl` must be a `logos/` key the caller uploaded.
GET/POST `/departments` **PUT/DELETE** `/departments/:id`	`{ name*, faculty, headName, code, description }`	Department name required. `code` unique per institution (e.g. CS-100).
GET `/members`	`role=tutor|student&q&departmentId&page&limit`	Paginated list with `inviteStatus`.
POST `/members`	`{ role, name*, email*, departmentIds[], designation, autoInvite }`	Duplicate email in the same institution returns 409. `autoInvite:true` sends the invitation email immediately.
POST `/members/import`	CSV (`name,email,department,designation`; `;` separates several departments) or JSON rows	`{ imported, skipped:[{ row, reason }] }`. Validate email format, dedupe by email, cap rows (e.g. 5,000), process asynchronously above a threshold.
POST `/members/:id/invite`	-	Resend invitation; 429 if resent too often.
DELETE `/members/:id`	-	Removes a draft member; if already registered, detaches instead of deleting the user.
POST `/subscription/checkout`	`{ plan, billing, method:"card"|"bank_transfer" }`	`{ reference, checkoutUrl, amount, currency, discount, total }`. **Card details must never reach our API** (see B4).
GET `/subscription`	-	Plan, status, period, invoices.
GET `/dashboard`	-	Stats (total students, total teachers, active courses, growth), active programs, weekly analytics, recent activity.

Gloxad reviewer portal (base path `/api/admin`; role `admin` with partner-review permission)

Endpoint	Request	Response and rules
GET `/institution-applications`	`status`, `q` (name, email, representative, type), `sort=submittedAt:desc`, `page`, `limit`	Rows for the table: `_id`, `referenceId`, `institutionName`, `email`, `representative.name`, `type`, `students`, `submittedAt`, `status`. Plus `pagination`.
GET `/institution-applications/:id`	-	Everything in the review drawer: institution, representative, expected students, requested features, message, documents, plan, notes, last-updated-by.
PUT `/institution-applications/:id`	`{ status?, plan?, notes? }`	Status select, plan select and internal notes. Writes a ReviewEvent. Notes are staff-only.
POST `/institution-applications/:id/approve`	-	Sets `approved`, creates the Institution, emails the activation link. Returns `{ application, invitation:{ sentTo, expiresAt } }`. Idempotent; second call returns 409.
POST `/institution-applications/:id/reject`	`{ reason? }`	Sets `rejected`, emails the representative.
POST `/institution-applications/:id/request-info`	`{ message }`	Sets `under_review`, emails the question.
POST `/institution-applications/:id/resend-invite`	-	Only when `approved`; otherwise 409. (UI warns "Approve the institution first".)
GET `/institution-applications/:id/documents/:docId/download`	-	`{ url }` signed GET, 5-minute expiry. Log the download.
GET `/partners/overview`	`range=30|90`	KPIs (total institutions, active students, monthly revenue, growth rate), applications-per-day series, recent activity.
GET `/partners/summary`	-	Active institutions (+% vs last month), pending approvals, approval success rate (approved and rejected percent).
GET `/partners/geo`	-	Counts by country with coordinates or ISO codes (map pins).
GET `/partners/growth`	`months=7`	New partners per month.
GET/POST/PUT/DELETE `/staff`	-	Reviewer staff and permissions. **DECISION** screen not designed.

B3. Rules the server must own

- **Application status transitions:** `pending` -> `under_review` -> `approved` | `rejected`. Approved and rejected are final unless an admin reopens (log it). The reviewer UI offers all four values in a select, so enforce the allowed transitions server-side.
- **Pricing (never trust the client):** Premium Growth **$249/month**: up to 500 students, 10 departments, advanced analytics, email support. Enterprise Scalability **$599/month**: unlimited students, dedicated account manager, custom SSO/SAML, priority 24/7 support, white-labeling. **Annual billing = 12 x monthly minus 20%** (Enterprise annual: $7,188.00 - $1,437.60 = **$5,750.40**; Premium annual: $2,988.00 - $597.60 = $2,390.40). "Taxes calculated at checkout."
- **Seat and department limits** enforced when adding members and departments (Premium: 500 students, 10 departments; Enterprise: unlimited).
- **Activation token:** single-use, 7-day expiry, bound to the representative's email. **Password policy:** minimum 12 characters (the UI already enforces this; enforce it here too).
- **Logo:** SVG, PNG, JPG or GIF. The UI states a maximum of 800x800 px and recommends at least 512x512 px, and rejects files over 2 MB. Re-validate type, size and (for SVG) sanitise to strip scripts.
- **Bulk import:** header names are `name,email,department,designation` (case-insensitive; `Professional Email` and `Full Name` are accepted aliases). Quoted cells and commas inside quotes must parse.
- **30-day money-back guarantee** is promised on the subscription screen: define the refund flow and who can trigger it.

B4. Payments DECISION

The subscription screen shows a card form (name, number, expiry, CVV) and a Bank Transfer tab. **The front end sends no card data anywhere today.** To go live without taking on card-handling obligations, use the provider's hosted checkout or tokenised fields (Paystack is already used for course checkout via `/api/enrollments/checkout` and `/api/webhooks/paystack`). Please confirm: provider for institution subscriptions, **billing currency** (the design shows USD; course prices elsewhere are in naira), recurring subscriptions or invoices, and how a bank-transfer payment is confirmed (manual by staff or reference matching). A signed webhook must activate the institution; the front end only polls `GET /me/subscription`.

B5. Emails

Trigger	To	Content
Application received	Representative	Reference ID, review time (24-48 hours), tracking link
Approved	Representative	Activation link (single use, 7 days), what happens next
Rejected / more information requested	Representative	Reason or question, how to reply
Member invited (tutor or student)	Member	Institution name, role, sign-up link tied to the invitation
Payment received / failed	Institution admin	Receipt, plan, next renewal, retry link
New application	Reviewer team	Optional notification with link to the drawer

Part C. Trimester LMS NEW

Front-end routes (all behind sign-in): `/lms` (dashboard, different per role), `/lms/curriculum` (daily lesson), `/lms/live`, `/lms/assignments`, `/lms/community`, `/lms/certificates` and `/lms/quiz`. Students, instructors and admins each see their own dashboard at `/lms`.

C0. Access rules

- **Today the front end lets any signed-in user open `/lms` and shows demo data.** The API must decide who sees what: a student only sees cohorts they are enrolled in, an instructor only cohorts they teach, an admin everything. Return 403 for anything else.
- `GET /api/lms/me` **NEW** -> `{ role, cohorts:[{ _id, name, trimester, programTitle, currentWeek }], activeCohortId }`. The front end will call this first to pick the cohort and to show a proper "You are not enrolled yet" state.
- Trimester courses already exist as normal courses (`GET /api/courses?courseType=trimester`) and students enrol through `/api/enrollments/*`. The LMS should be unlocked by an **active enrolment in a cohort**, not by role alone.

C1. Data model

Entity	Key fields
Trimester (term)	`number` (1-3), `title`, `startDate`, `endDate`, `weeks`, `enrollmentClosesAt` (the public site shows "Enrollment Closes: March 10, 2026")
Cohort	`name` ("Cohort B"), `courseId`, `trimesterId`, `instructorIds[]`, `studentIds[]`, `capacity` (e.g. 45), `timezone`, `status`
Module / Lesson	Module: `courseId`, `order`, `title`, `week`. Lesson: `moduleId`, `order`, `title`, `type` (`video`, `reading`, `quiz`), `video{ provider, assetId, durationSec }`, `notes` (sanitised HTML or markdown), `objectives[]`, `attachments[{ name, key, size }]`, `estimatedMinutes`
LessonProgress	`userId`, `lessonId`, `status`, `watchedSeconds`, `completedAt`, `objectivesDone[]`
Quiz / Question	Quiz: `moduleId`, `title`, `timeLimitMinutes` (15), `passMarkPercent` (70). Question: `type` (`multiple_choice`), `prompt`, `hint`, `options[{ _id, text }]`, `correctOptionId` and `explanation` (**never sent before the answer is checked**)
QuizAttempt	`quizId`, `userId`, `startedAt`, `expiresAt`, `answers[{ questionId, optionId, flagged, checkedAt }]`, `score`, `status` (`in_progress`, `submitted`, `expired`)
Assignment / Submission	Assignment: `moduleId`, `title`, `summary`, `description`, `dueAt`, `rubric[{ criterion, points }]`, `maxFileMB` (50), `allowedTypes`, `allowRepoLink`. Submission: `assignmentId`, `userId`, `fileKey` or `repoUrl`, `submittedAt`, `status` (`submitted`, `pending_review`, `graded`), `grade{ score, max, feedback, rubricScores[], gradedBy, gradedAt }`
LiveSession	`cohortId`, `title`, `instructorId`, `startsAt`, `endsAt`, `status` (`scheduled`, `live`, `ended`), `provider`, `roomId`, `recordingUrl`
Attendance	`sessionId`, `userId`, `status` (`present`, `absent`, `excused`), `joinedAt`, `leftAt`
Chat, Poll	ChatMessage: `sessionId`, `authorId`, `text`, `createdAt`. Poll: `sessionId`, `question`, `options[]`, `votes[{ userId, optionIndex }]`, `status`
Channel / Post / Reply	Channel: `cohortId`, `slug`, `name` (General, Q&A, Project Showcase, Study Groups), `isNew`. Post: `channelId`, `authorId`, `tag` (`Discussion`, `Showcase`, `Q&A`), `title`, `body`, `likes[]`, `replyCount`, `createdAt`. Reply: `postId`, `authorId`, `body`
Announcement	`cohortId`, `title`, `body`, `publishedAt`, `authorId`
Certificate	`certificateId` (public, unguessable), `userId`, `courseId`, `title`, `subtitle`, `badge` (`Gloxad Certified`, `Specialization`), `issuedAt`, `pdfKey`, `revokedAt`

C2. Existing routes to reuse VERIFY

These are documented and cover part of the LMS. Please confirm they return the data the screens need, or extend them instead of duplicating.

Existing route	Used for
`GET /api/courses?courseType=trimester`, `GET /api/courses/:slugOrId`	Trimester catalogue. For enrolled students the course payload must include unshielded lessons.
`POST /api/enrollments/enroll/:courseId`, `.../checkout`, `GET .../my-courses`, `GET .../check/:courseId`	Enrolment and access check.
`POST /api/learning/:courseId/progress`	Lesson progress (watched, completed).
`POST /api/learning/:courseId/quiz/:quizId/submit`	Quiz scoring. The new quiz screen needs the timed-attempt and per-question check flow in C3.
`GET/POST /api/learning/:courseId/questions` (+ `/:questionId/reply`)	Lesson Q&A. Candidate for the lesson page "Class Discussion" (add likes and the `Instructor` badge).
`GET/POST /api/learning/:courseId/announcements`	Dashboard announcements.
`GET /api/learning/:courseId/certificate`, `GET /api/learning/verify-certificate/:certificateId`	Certificate and public verification.
`/api/messages/*`, `/api/notifications`, `POST /api/notifications/mark-read`	Direct messages (mail icon) and notifications (bell icon).
`GET /api/admin/users`, `GET /api/admin/users/:id`	Admin user table. Add `status` (`active`/`suspended`) and `lastLoginAt`.

C3. New endpoints by screen (all `/api/lms`, auth required)

Student dashboard (`/lms`)

Endpoint	Request	Response and rules
GET `/dashboard/student`	`cohortId`	`{ cohort{ name, programTitle, week }, progress{ overall, week, assignments{ done, total }, attendance }, upNext{ lessonId, title, estimatedMinutes }, upcoming[{ type, title, dueAt, refId }], announcements[{ _id, publishedAt, title, excerpt }] }`. `type` is `reading`, `assignment`, `live_class` or `milestone`.

Curriculum and lesson (`/lms/curriculum`)

Endpoint	Request	Response and rules
GET `/cohorts/:cohortId/curriculum`	-	Modules with lessons and per-lesson status (`not_started`, `in_progress`, `completed`).
GET `/lessons/:lessonId`	-	`{ trimester, module, title, video{ playbackUrl or token, durationSec }, notes, objectives[{ text, done }], attachments[{ name, size, url (signed) }], progressPercent, next[{ title, meta, type, id }], previousId }`. Video URL must be signed or token-gated.
PUT `/lessons/:lessonId/progress`	`{ watchedSeconds, completed?, objectivesDone[] }`	Idempotent; recalculates the dashboard percentages.
GET/POST `/lessons/:lessonId/comments`	`{ body }`	Threaded: `replies[]`, `likesCount`, `likedByMe`, author `role` ("Instructor" badge).
POST `/comments/:id/replies`, **POST/DELETE** `/comments/:id/like`	`{ body }`	Like is a toggle per user.

Quiz (`/lms/quiz`)

Endpoint	Request	Response and rules
GET `/quizzes/:quizId`	-	`{ title, programTitle, moduleName, timeLimitMinutes, questionCount, passMarkPercent }`. No answers.
POST `/quizzes/:quizId/attempts`	-	`{ attemptId, startedAt, expiresAt, questions:[{ _id, type, prompt, hint, options:[{ _id, text }] }] }`. Answers are not included. One in-progress attempt per user; define retake rules **DECISION**.
POST `/quiz-attempts/:attemptId/answers`	`{ questionId, optionId, flagged?, check? }`	Saves the choice. With `check:true` (the "Check Answer" button) returns `{ correct, correctOptionId, explanation }` and **locks** that answer. 409 if the attempt has expired.
POST `/quiz-attempts/:attemptId/submit`	-	`{ score, total, percent, passed }`. The timer is enforced **server-side**; an expired attempt is auto-submitted. The UI counts down from `expiresAt`.

Assignments (`/lms/assignments`)

Endpoint	Request	Response and rules
GET `/assignments`	`cohortId`, `filter=all|upcoming|past`	`[{ _id, module, title, summary, dueAt, status, progress, grade? }]`. `status`: `not_started`, `submitted`, `pending_review`, `graded`.
GET `/assignments/:id`	-	Adds `description`, `rubric[{ criterion, points }]`, `maxFileMB`, `allowedTypes`, `mySubmission`.
POST `/assignments/:id/submissions`	`{ fileKey }` **or** `{ repoUrl }`, optional `comment`	201 `{ submission{ status:"pending_review", submittedAt } }`. `repoUrl` must be `https://github.com/<owner>/<repo>`. File: 50 MB, zip, pdf or source. Define late-submission and resubmission policy **DECISION**.
GET `/instructor/grading-queue`	`cohortId`	`{ pending, items:[{ submissionId, module, assignmentTitle, student{ name, avatarUrl }, submittedAt, overdue }] }`.
PUT `/submissions/:id/grade`	`{ score, feedback, rubricScores[] }`	Sets `graded`, notifies the student.

Live class (`/lms/live`)

Endpoint	Request	Response and rules
GET `/live-sessions`	`cohortId`, `status`	Sessions plus `next{ title, startsAt }` for the header countdown ("Starts in 08:45").
POST `/live-sessions` (instructor) **PUT/DELETE** `/:id`	`{ cohortId, title, startsAt, endsAt, description }`	The "Schedule Live Class" button on the instructor dashboard.
POST `/live-sessions/:id/join`	-	`{ provider, roomToken, serverUrl, role }`. Short-lived, only for enrolled users; records attendance.
POST `/live-sessions/:id/polls` (instructor) **POST** `/polls/:pollId/votes`	`{ question, options[] }` / `{ optionIndex }`	One vote per user. Results shown to the instructor; the UI tells students results are shared when the poll closes.
GET `/live-sessions/:id/attendance` (instructor) **PUT** `.../:studentId`	`{ status }`	Used by the roster ("Excused").

Real time (socket.io, namespace `/live`, JWT in `auth.token`): `chat:message`, `hand:raise`, `hand:lower`, `poll:launched`, `poll:closed`, `participants:update`, `session:ended`. Persist chat so late joiners and moderators can see it. The screen has chat, polls, a people list (24 in the design), raise-hand, mic, camera, screen share and leave.

Community (`/lms/community`)

Endpoint	Request	Response and rules
GET `/channels`	`cohortId`	`[{ _id, slug, name, postCount, isNew }]`. Seed General, Q&A, Project Showcase, Study Groups for every cohort.
GET `/posts`	`channelId` or `cohortId`, `page`, `limit`	`[{ _id, author{ _id, name, avatarUrl, role }, channel, tag, title, body, likesCount, likedByMe, repliesCount, createdAt }]`. The General feed shows posts from all channels.
POST `/posts`	`{ channelId, title, body, tag }`	Title up to 120, body up to 5,000 characters; sanitise. Rate-limit.
POST/DELETE `/posts/:id/like`, **GET/POST** `/posts/:id/replies`	`{ body }`	The UI has a Replies button but no thread view yet.
DELETE `/posts/:id`, **POST** `/posts/:id/report`	-	Moderation by the author, instructors and admins.

Certificates (`/lms/certificates`)

Endpoint	Request	Response and rules
GET `/certificates/me`	-	`{ inProgress:[{ courseId, trimester, title, expectedCompletion, progressPercent }], earned:[{ _id, certificateId, title, subtitle, badge, completedAt }] }`.
GET `/certificates/:id/pdf`	-	`{ url }` signed. The button is "View PDF". Generate the PDF server-side with a QR code to the verification page.
Issuance (internal)	-	Issue automatically when the course is complete (required lessons done, quizzes passed, assignments graded). Admin can revoke or reissue.

"Share to LinkedIn" opens LinkedIn's add-certification form pre-filled with the title, organisation (Gloxad Academy) and issue month and year. If you return `certificateId` and a public verification URL, the front end will also pass the credential ID and URL.

Instructor cohort overview (`/lms`, role instructor)

Endpoint	Request	Response and rules
GET `/instructor/cohorts`	-	Cohorts taught.
GET `/instructor/cohorts/:id/overview`	-	`{ trimesterTitle, stats{ avgCompletion, activeStudents{ now, total, deltaThisWeek }, avgGrade, gradeTrend }, moduleCompletion:[{ moduleId, title, completionPercent }], gradingQueue{ pending, items[] } }`.
GET `/instructor/cohorts/:id/roster`	`sessionId`, `q`, `status`, `page`, `limit`	`[{ student{ _id, name, email, avatarUrl }, attendance, progressPercent, lastActiveAt }]` + pagination (the UI has "Load More Students").

Admin analytics (`/lms`, role admin)

Endpoint	Request	Response and rules
GET `/admin/analytics`	`range`	`{ totalRevenue{ amount, currency, deltaPercent, since }, enrollments{ count, deltaPercent }, completionRate{ percent, note }, platformHealth{ uptimePercent, window }, enrollmentTrends:[{ period, count }] }`.
GET `/admin/certificates/recent`	`limit`	`[{ title, issuedToName, issuedAt }]`.
GET/PUT `/api/admin/users`, `/:id`	`role`, `status`, `q`, `page`, `limit`; PUT `{ role?, status? }`	User table with search, role filter, suspend/activate and edit. Show "Showing 1 to 3 of 45 entries" from `pagination.total`.

C4. Rules the server must own

- **Progress:** define how "overall program progress" (65% in the design), "Week 2: 40%", "Assignments 5/8" and "Attendance 100%" are computed. Proposal: overall = completed required items (lessons, quizzes, assignments) divided by all required items; week = the same for items due this week; attendance = sessions attended divided by sessions held, with excused sessions removed from the denominator.
- **Quiz:** pass mark 70%; correct answers only revealed after a check or submit; time limit enforced by the server using `expiresAt`.
- **Time zones:** store UTC, return ISO strings, and give each cohort a time zone for labels such as "Due tomorrow at 5:00 PM" and "Friday, EOD".
- **Grades:** a letter scale for the cohort average ("B+"), documented thresholds.
- **Notifications** (in-app and email): assignment due in 24 hours, assignment graded, live class starting in 15 minutes, new announcement, reply to your post or comment, certificate issued.

C5. Decisions needed

- **Video provider for live classes** (LiveKit, Daily, Agora, Zoom Video SDK or Jitsi): cost, recording, and latency from Nigeria. This is the largest single piece of work in the LMS.
- **Lesson video hosting:** S3 plus CDN with signed URLs, or a video service (Mux, Cloudflare Stream, Vimeo).
- **Cohorts and pricing:** how a student is placed in a cohort, trimester pricing in naira and dollars, and whether partner institutions get their own cohorts and isolated data.
- **Retakes and late work:** quiz attempts allowed, late-submission policy, resubmission.

Part D. Cross-cutting requirements

D1. Roles and permissions

Role	Can do
`student`	Own profile and progress; enrolled cohorts only; submit work, take quizzes, post in the forum, view own certificates.
`instructor` (approved)	Cohorts they teach: grade, schedule and run live classes, post announcements, view roster and analytics.
`admin` (platform)	Everything above for all cohorts, user management, course approval, finance, platform settings.
Partner reviewer **NEW**	A permission on `admin` (not a separate login type): list and decide partner applications, view documents.
`institution_admin` **NEW**	Manage their own institution only: branding, departments, members, subscription, dashboard.
Institution tutor or student **NEW**	Invited by an institution; register as normal users with an `institutionId`.

D2. Security and privacy

- **Validate everything on the server.** The front end validates passwords, emails, file types and sizes for convenience only.
- **Rate limits:** sign-in and password reset (for example 5 per minute per IP), public partner application (5 per hour per IP), invitations, forum posts, uploads.
- **Files:** enforce type and size through the presigned policy, scan submissions and documents for malware, serve private files only through signed URLs.
- **Card data must not touch our servers** (see B4). Verify provider webhooks by signature.
- **Personal data:** student and institution names, emails, phone numbers and submissions. Agree retention, export and deletion on request, and publish the matching policy (the site footer links to Terms and Privacy pages that do not exist yet).
- **Audit log** for admin and reviewer actions (who, what, when). Logs must not contain tokens, passwords or full payment details.
- **Tokens:** keep the JWT secret in the environment, bump `tokenVersion` on password change and logout-everywhere, and never accept `demo-token-*` values (the front end uses those only in local development).
- **Multi-tenant isolation** for institutions: every institution-scoped query must filter by `institutionId`. The partner landing page promises "multi-tenant data isolation"; this must be true.

D3. Uploads

Extend `POST /api/upload/presigned-url` to accept the folders in A1 and return `{ uploadUrl, fileUrl, key, expiresIn }` as today. For private folders return only `key` (no public `fileUrl`) and provide `GET /api/upload/signed-url?key=` for authorised reads.

D4. Email

A transactional provider with templates for everything in B5 plus: assignment graded, live class reminder, certificate issued, announcement digest. Include plain-text versions, an unsubscribe policy for non-essential mail, and deliverability setup (SPF, DKIM, DMARC) on the sending domain.

D5. Conventions

- Consistent pagination and `{ error, fields }` errors across **all** new routes (see section 1).
- Add database indexes for every filter used above (for example `status`, `submittedAt`, `cohortId`, `userId`, `email`) and unique indexes where stated (member email per institution, department code per institution).
- Idempotency for webhooks, approve, invite and certificate issuance.
- Soft-delete or archive instead of hard delete for anything a student or institution may need to recover or an auditor may need to see.

D6. Environments and configuration

Variable	Where	Notes
`BACKEND_URL`	Web app	Target of the `/api/*` proxy.
`NEXT_PUBLIC_BACKEND_URL`	Web app	Used directly by socket.io (messaging and live class). Must allow WebSocket upgrades.
`NEXT_PUBLIC_API_URL`	Web app (optional)	Skip the proxy and call the API directly (needs CORS).
`NEXT_PUBLIC_REQUIRE_STUDENT_EMAIL_VERIFICATION`	Web app	Set to `false` only if the API does not email students a code.
`NEXT_PUBLIC_ENABLE_DEMO_LOGIN`	Web app	**Must be unset in production.** Demo accounts are accepted entirely in the browser.
`CORS_ORIGINS`	API (suggested)	Comma-separated allow-list (see A2).
Bucket, region, credentials	API	See A1.
Email, payment and video provider keys	API	Per provider chosen in B4 and C5.

D7. Documentation deliverable

Please publish an **OpenAPI 3 JSON** (for example at `/api/openapi.json`) and a Postman collection, kept in step with the reference page. The reference page today is HTML only, which makes it impossible to generate types or run contract tests.

D8. Observability

- Split `/api/health` into liveness (process up) and readiness (database and storage reachable).
- Feed an external uptime monitor and expose the monthly uptime percentage to `GET /api/lms/admin/analytics` so "Platform Health" is a measured number, not a fixed one.
- Error tracking, request IDs in logs, and database backups with a tested restore.

Part E. Suggested delivery order and acceptance checks

Phase	Scope	Done when
0. Unblock (this week)	A1 bucket and CORS, A2 API CORS, A5 staging + credentials + OpenAPI, A3/A4 confirmations	A profile photo uploads from the browser and displays; every seeded image loads; the production origin can call the API; the front end can test signed-in routes.
1. Partner applications	B2 public application, reviewer list/detail/decide, approve with emailed activation, B5 emails, audit events	A request submitted on the site appears in the reviewer table; approving sends a working activation link; every decision is in the audit trail.
2. Institution onboarding and billing	Activation, branding, departments, members and CSV import, plans, checkout and webhook, institution dashboard	A new institution completes all six steps from a second device, pays, and lands on a dashboard with real numbers.
3. LMS core	`/lms/me`, cohorts and enrolment gate, student dashboard, curriculum and progress, quiz attempts, assignments and grading	An enrolled student sees real progress; a non-enrolled user gets 403; a quiz is timed and scored on the server; a submission reaches the grading queue.
4. Community, certificates, analytics	Forum, certificate issuance and PDF, instructor overview and roster, admin analytics and users	A completed course issues a verifiable certificate; instructor and admin numbers match the database.
5. Live classes	Provider integration, join tokens, attendance, socket events, polls, scheduling	A scheduled class goes live, students join with video and chat, attendance appears in the roster.

Acceptance checklist for the first hand-over

- Bucket exists; `GET` on a seeded image URL returns 200; presigned `PUT` from `http://localhost:3000` and the production origin succeeds.
- `OPTIONS` preflight from the production origin returns the right `Access-Control-Allow-*` headers on `/api/auth/signin` and an authenticated route.
- Staging API reachable with seed users for each role and a shared OpenAPI file.
- `POST /auth/refresh` returns a new token for a recently expired one.
- `POST /admin/categories` stores `imageUrl` and returns it from `GET /categories`.

Appendix A. Screen to API map

Screen (route)	Data and actions	Endpoints
Partner landing (`/institution`)	Static marketing content; Get Started and Request Demo	None yet; Request Demo should create an application (B2).
Activate (`/institution/activate`)	Set password (min 12)	`POST /institutions/activate`
Onboarding: logo	Upload logo (svg, png, jpg, gif; max 800x800)	`POST /upload/presigned-url` (folder `logos`), `PUT /institutions/me/branding`
Onboarding: departments	Name, faculty, head, code, description; add and remove	`/institutions/me/departments`
Onboarding: tutors / students	Manual add, CSV import, template download, auto-invite toggle, draft list	`/institutions/me/members`, `.../members/import`
Onboarding: subscription	Plan, monthly or annual, payment method, order summary	`GET /institutions/plans`, `POST .../subscription/checkout`
Onboarding: branding	Display name, colour, tagline, live preview	`PUT /institutions/me/branding`
Application submitted	Reference ID, review time, Track Request	`GET /institutions/applications/:referenceId`
Institution dashboard (`/institution/dashboard`)	Stats, active programs, analytics chart, quick actions, activity	`GET /institutions/me/dashboard`
Reviewer dashboard (`/admin/partners`)	KPIs, 30/90-day applications chart, recent activity, shortcuts	`GET /admin/partners/overview`
Applications (`/admin/partners /applications`)	Search, filter, sort, paging, stats, map, growth chart	`GET /admin/institution-applications`, `/partners/summary`, `/geo`, `/growth`
Review drawer and approval modal	Details, documents, status, plan, notes, approve, reject, request info, invite	`GET/PUT .../:id`, `POST .../approve|reject|request-info|resend-invite`, document download
LMS student dashboard	Progress, up next, upcoming, quick access, announcements	`GET /lms/dashboard/student`
LMS lesson	Video, notes, objectives, attachment, discussion, up next	`GET /lms/lessons/:id`, `PUT .../progress`, `.../comments`
LMS quiz	Timed multiple choice, check answer, flag, results	`POST /quizzes/:id/attempts`, `/quiz-attempts/:id/answers|submit`
LMS assignments	Filter, detail, rubric, upload or GitHub link	`GET /assignments`, `POST /assignments/:id/submissions`
LMS live class	Video, controls, chat, polls, people, raise hand, countdown	`/live-sessions`, join token, socket `/live`
LMS community	Channels, posts, likes, new post, share	`/channels`, `/posts`, likes
LMS certificates	In progress, earned, view PDF, share to LinkedIn	`GET /certificates/me`, `/certificates/:id/pdf`
LMS instructor overview	Analytics, grading queue, roster and attendance	`/instructor/cohorts/:id/overview`, `/roster`, `/grading-queue`
LMS admin analytics	Revenue, enrolments, completion, health, trends, user table	`GET /lms/admin/analytics`, `/admin/users`

Appendix B. Example payloads

B.1 Partner application: submit

```
POST /api/institutions/applications
{
  "institutionName": "St. Andrews Polytechnic",
  "type": "Tertiary Institution",
  "website": "www.standrews-poly.edu.ng",
  "address": { "line1": "42 Victoria Way", "city": "Lagos", "country": "NG" },
  "representative": { "name": "Dr. Samuel Adebayo", "role": "Director of Admissions",
                      "email": "s.adebayo@standrews.edu", "phone": "+234 801 234 5678" },
  "expectedStudents": "2,500 - 5,000",
  "requestedFeatures": "Student portal, automated grading, multi-campus management",
  "message": "We are looking to digitize our admission workflow.",
  "documentKeys": ["documents/2026/10/cac-registration.pdf"]
}
-> 201 { "application": { "_id": "...", "referenceId": "GLX-2026-9012", "status": "pending",
                          "submittedAt": "2026-10-07T09:12:44.000Z" } }
```

B.2 Approve an application

```
POST /api/admin/institution-applications/:id/approve
-> 200 { "application": { "status": "approved", "updatedBy": "reviewer_2", "updatedAt": "..." },
         "invitation":  { "sentTo": "s.adebayo@standrews.edu", "expiresAt": "2026-10-14T09:15:00.000Z" } }
-> 409 { "error": "Application is already approved" }
```

B.3 Student dashboard

```
GET /api/lms/dashboard/student?cohortId=...
{
  "cohort": { "name": "Cohort B", "programTitle": "Full-Stack Web Development", "week": 2 },
  "progress": { "overall": 65, "week": 40, "assignments": { "done": 5, "total": 8 }, "attendance": 100 },
  "upNext": { "lessonId": "...", "title": "React State Management", "estimatedMinutes": 45 },
  "upcoming": [
    { "type": "reading",    "title": "Advanced Hooks Concept", "dueAt": "2026-10-07T22:59:00.000Z", "refId":
"..." },
    { "type": "assignment", "title": "Build a Todo App",       "dueAt": "2026-10-08T16:00:00.000Z", "refId":
"..." },
    { "type": "live_class", "title": "Weekly Code Review",     "dueAt": "2026-10-09T09:00:00.000Z", "refId":
"..." }
  ],
  "announcements": [ { "_id": "...", "publishedAt": "2026-10-07T08:00:00.000Z",
                       "title": "Class schedule updated", "excerpt": "Thursday's live session has been
moved..." } ]
}
```

B.4 Quiz: start, check, submit

```
POST /api/lms/quizzes/:quizId/attempts
-> { "attemptId": "a1", "startedAt": "...", "expiresAt": "...(+15 min)",
     "questions": [ { "_id": "q3", "type": "multiple_choice",
        "prompt": "In the context of Neural Networks, what is the primary purpose of the activation function?",
        "hint": "Consider the role it plays in allowing the network to learn complex patterns.",
        "options": [ { "_id": "o1", "text": "To initialize the weights..." }, { "_id": "o2", "text": "To
introduce non-linearity..." } ] } ] }

POST /api/lms/quiz-attempts/a1/answers   { "questionId": "q3", "optionId": "o2", "check": true }
-> { "correct": true, "correctOptionId": "o2", "explanation": "Without non-linearity, stacked layers collapse
into one linear map." }

POST /api/lms/quiz-attempts/a1/submit
-> { "score": 9, "total": 10, "percent": 90, "passed": true }
```

B.5 Assignment submission

```
POST /api/lms/assignments/:id/submissions
{ "repoUrl": "https://github.com/alex/api-integration" }          // or { "fileKey":
"submissions/2026/10/alex.zip" }
-> 201 { "submission": { "status": "pending_review", "submittedAt": "2026-10-07T10:05:00.000Z" } }
-> 400 { "error": "Invalid repository link", "fields": { "repoUrl": "Use https://github.com/<owner>/<repo>" } }
```

B.6 Live class: join and socket events

```
POST /api/lms/live-sessions/:id/join
-> { "provider": "livekit", "serverUrl": "wss://...", "roomToken": "eyJ...", "role": "student" }

socket.io  namespace /live   auth: { token: <JWT> }
  emit  chat:message   { sessionId, text }
  on    chat:message   { id, author: "David L.", time: "2026-10-07T10:14:00.000Z", text }
  emit  hand:raise     { sessionId }            on  hand:raise  { userId, name }
  on    poll:launched  { pollId, question, options: ["O(V)", "O(V + E)", "O(V x E)", "O(log V)"] }
  on    participants:update { count: 24 }
```

Appendix C. Enumerations

Name	Values
Application status	`pending`, `under_review`, `approved`, `rejected`
Plan / billing	`Premium` or `Enterprise` / `monthly` or `annual`
Invite status	`not_invited`, `pending`, `accepted`, `expired`
Assignment status	`not_started`, `submitted`, `pending_review`, `graded`
Attendance	`present`, `absent`, `excused`
Live session status	`scheduled`, `live`, `ended`
Post tag	`Discussion`, `Showcase`, `Q&A`
Certificate badge	`Gloxad Certified`, `Specialization`
Upcoming item type	`reading`, `assignment`, `live_class`, `milestone`
User status	`active`, `suspended`

Appendix D. Questions for the backend engineer

1. Which flow in B0 do we adopt, and who owns the partner request form copy and fields?
2. What are the bucket name, region and CDN domain, and when can the seed images be restored?
3. Which payment provider and currency for institution subscriptions, and does it support recurring billing and bank-transfer confirmation?
4. Which video provider for live classes, and who owns recording and retention?
5. Are cohorts modelled on top of the existing Course and Enrollment collections, or as a new structure?
6. Can the existing `/learning/*` routes carry lesson discussion, announcements and quiz scoring, or should the LMS get its own?
7. Can you share staging credentials and an OpenAPI file this week, and what is the expected turnaround for Phase 0?
8. Which of the duplicate response fields from A6 will be canonical, and by what date can the other spelling be dropped?